

Module 04

Introduction to Programming Language with AI (JavaScript + Node.js)

SUBMITTED BY

Student Name: MD. MAHFUZUR RAHMAN

ID: 01402806910

Batch: 21

ASSIGNMENT TOPICS & QUESTIONS

Q1. JavaScript Coding Practice

Write programs for the following:

- Check whether a number is even or odd.
- Find the largest of three numbers.
- Reverse a string.
- Count vowels in a string.
- Remove duplicate values from an array.

Q2. Introduction to Node.js

Explain:

- What is Node.js?
- How does Node.js work?
- Advantages of Node.js for QA Automation
- Difference between Browser JavaScript and Node.js

Contents

Q2. Introduction to Node.js	3
1. What is Node.js?	3
2. How does Node.js work?	3
3. Advantages of Node.js for QA Automation	3
4. Difference between Browser JavaScript and Node.js	4

Q2. Introduction to Node.js

1. What is Node.js?

Node.js is an open-source, cross-platform JavaScript runtime environment built on Google Chrome's V8 JavaScript engine. It allows developers to execute JavaScript code outside of a browser environment, enabling server-side scripting, CLI tool development, and network applications.

- **V8 Engine Powered:** Translates JavaScript code directly into native machine code, providing exceptional execution speed.
- **Full-Stack Capability:** Enables JavaScript to be used across both frontend and backend development, unifying the entire codebase under a single programming language.

2. How does Node.js work?

Node.js operates on a Single-Threaded, Event-Driven, Non-Blocking I/O (Input/Output) architecture. This architecture allows Node.js to handle thousands of concurrent client connections efficiently without creating multiple threads for each request.

- **Single-Threaded Model:** Executes application logic on a single main thread, drastically reducing memory overhead and avoiding thread synchronization complexities.
- **Event Loop:** The central engine of Node.js. It continuously monitors the execution queue and dispatches non-blocking tasks. When operations complete, their corresponding callback functions are pushed to the queue for execution.
- **Non-Blocking I/O:** Heavy operations like file system access, network calls, or database queries are delegated to underlying C++ worker threads (via libuv). Node.js immediately continues processing next tasks without blocking waiting for completion.

3. Advantages of Node.js for QA Automation

Node.js has become a leading ecosystem for Quality Assurance (QA) and Automated Software Testing due to several distinct technical advantages:

- **Native Modern Automation Tooling:** Industry-standard web and mobile automation frameworks such as Playwright, Cypress, WebDriverIO, and Appium run natively on Node.js.
- **Unified Development Stack:** QA engineers and developers write code in the same language (JavaScript / TypeScript), fostering better collaboration, code reviews, and shared utilities.
- **Fast Test Suite Execution:** The non-blocking event loop and V8 engine ensure high-speed test script execution, reducing overall pipeline runtimes.
- **NPM Ecosystem Support:** Access to millions of packages via NPM (Node Package Manager) for test runners (Mocha, Jest), assertion libraries (Chai), reporting, and API mocking.
- **CI/CD Pipeline Integration:** Lightweight, easy installation in Docker containers and continuous integration platforms like GitHub Actions, GitLab CI, and Jenkins.

4. Difference between Browser JavaScript and Node.js

Although both environments execute JavaScript, they serve different roles and provide access to distinct APIs:

Feature	Browser JavaScript	Node.js
Execution Environment	Runs inside a Web Browser (Chrome, Firefox, Safari).	Runs on the local operating system or server using Chrome's V8 engine.
Primary Goal	DOM manipulation, client-side UI rendering, and user interactivity.	Server-side logic, backend services, file handling, and test automation.
DOM / BOM Access	Full access to DOM window, document, and Web APIs.	No access to DOM window or document (they are undefined).
System Operations	Restricted access to local OS/file system due to security sandbox.	Direct access to OS features, file system (fs), and network modules.
Global Object	Global object is window.	Global object is global.
Module System	Uses ES Modules (import / export) natively.	Supports CommonJS (require) and ES Modules (import / export).